Large Language Model (LLM) for Standard Cell Layout Design Optimization

Chia-Tung Ho
NVIDIA Research
Santa Clara, CA, USA
chiatungh@nvidia.com

Haoxing Ren
NVIDIA Research
Austin, TX, USA
haoxingr@nvidia.com

2024 IEEE LLM Aided Design Workshop (LAD) | 979-8-3503-7608-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/LAD62341.2024.10691722

***Abstract*—Standard cells are essential components of modern digital circuit designs. With process technologies advancing toward 2*nm*, more routability issues have arisen due to the decreasing number of routing tracks, increasing number and complexity of design rules, and strict patterning rules. The state-of-the-art standard cell design automation framework is able to automatically design standard cell layouts in advanced nodes, but it is still struggling to generate highly competitive Performance-Power-Area (PPA) and routable cell layouts for complex sequential cell designs. Consequently, a novel and efficient methodology incorporating the expertise of experienced human designers to incrementally optimize the PPA of cell layouts is highly necessary and essential.**

High-quality device clustering, with consideration of netlist topology, diffusion sharing/break and routability in the layouts, can reduce complexity and assist in finding highly competitive PPA, and routable layouts faster. In this paper, we leverage the natural language and reasoning ability of Large Language Model (LLM) to generate high-quality cluster constraints incrementally to optimize the cell layout PPA and debug the routability with ReAct prompting. On a benchmark of sequential standard cells in 2*nm*, we demonstrate that the proposed method not only achieves up to 19.4% smaller cell area, but also generates 23.5% more LVS/DRC clean cell layouts than previous work. In summary, the proposed method not only successfully reduces cell area by 4.65% on average, but also is able to fix routability in the cell layout designs.

I. Introduction

Standard cells are essential components of modern digital circuit designs. As process technologies relentlessly advance toward 2*nm*, designing a cell with competitive Performance-Power-Area (PPA) while considering routability becomes increasingly challenging due to the decreasing number of routing tracks, increasing complexity of design rules, and strict patterning rules. The state-of-the-art standard cell design automation framework is capable of automatically designing standard cell layouts in advanced nodes, but it still struggles to generate highly competitive PPA and routable cell layouts for complex sequential cell designs. As a result, a novel and efficient methodology, leveraging the expertise of experienced human designers, to optimize the PPA and routability of cell layouts incrementally, has emerged as a critical need.

Recently, automated standard cell synthesis tools such as NVCell [1] and BonnCell [2], have been shown to generate high quality cell layouts on advanced technology nodes. Due to routability issues, one of the key challenges is that the generated placement for any given cell could be unroutable or unable to be routed without DRC errors. NVCell2 [3] develops a lattice graph routability model and successfully improves the routability in the advanced technology nodes. However, its performance is not scale to hundreds of transistors because the model inference needs to be performed for every action in the simulated annealing-based placement algorithm [1] and the cell-level metrics (i.e., cell width (CW) and total wirelength (TWL)) are compromised for routability. Ho *et al.* [4] proposed a transformer model-based cluster approach to generate high-quality device cluster constraints, which considers diffusion sharing/break, routability, and DRCs of routing metals in the layout of different technology nodes, and achieved better PPA, routability, and performance than [3] in the advanced nodes. However, selecting a good set of LVS/DRC clean layouts to train the transformer cluster model for optimizing PPA and routability of complex sequential cells together is quite challenging. This is because cells with routability issues typically have a larger cell width to reduce transistor pin density, while cells with a more compact layout could exacerbate routability issues. Additionally, there is a limited amount of LVS/DRC clean layouts available for training the transformer cluster model in the early development stage of cell library in a new technology node.

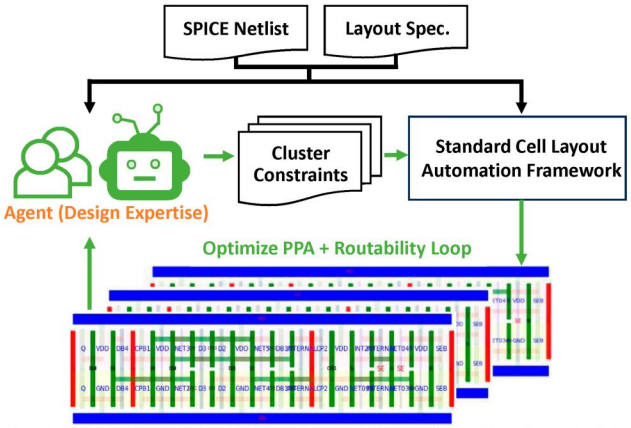


Fig. 1: An illustration of the PPA and routability optimization loop includes an agent with design expertise, adjusted cluster constraints, and a standard cell layout automation framework (i.e., NVCell).

An agent with designers' expertise can adjust and fine-tune device clustering constraints incrementally based on the netlist and the layout of the previous iteration to efficiently optimize PPA and routability together as shown in Figure 1. Lately, Large Language Models (LLMs) have shown great promise across various tasks in language understanding and interactive decision-making, incorporating reasoning and actions. In this paper, we leverage the natural language and reasoning ability of LLMs to adjust the device clustering constraints incrementally,

979-8-3503-7608-1/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:28:44 UTC from IEEE Xplore. Restrictions apply.

optimizing cell layout PPA and routability with guidance from designers' expertise and ReAct [5] prompting techniques. Our main contributions are as follows.

- We are the first to explore LLM for optimization in Electronic Design Automation (EDA) on an industrial-level benchmark. We propose a novel, and efficient LLM for standard cell layout design optimization methodology to generate high-quality cluster constraints to optimize the cell layout PPA and debug the routability with guidance of designers' expertise and ReAct [5] prompting techniques on an industrial technology node. The proposed methodology can improve the cell-level PPA and generate the device clusters incrementally considering netlist, previous cluster constraints, routability, and physical layout, simultaneously.
- We conduct holistic assessments and studies on the capabilities and domain knowledge of existing LLM on SPICE netlist language, cluster design constraint format, and physical layout description. Then, we automate the domain knowledge extraction with guidelines from designers' expertise for optimizing the PPA and routability together.
- The proposed novel LLM for standard cell layout design optimization methodology achieves up to 19.4% smaller cell area, and generates 23.5% more LVS/DRC clean cell layouts than a state-of-the-art baseline on a benchmark of sequential standard cells in industrial 2*nm* technology node.

The remaining sections are organized as follows: Section II demonstrates the study and assessment of existing LLM on understanding the netlist and standard cell layout domain knowledge. Section III describes our novel LLM for standard cell layout design optimization methodology. Section IV presents our main experiment. Section V concludes the paper.

II. Standard Cell Layout Design Domain Knowledge

We conduct assessments and studies on the capabilities and domain knowledge of existing LLMs of SPICE language format, device cluster constraints, and physical layout of standard cells.

A. SPICE Netlist Language

To assess the domain knowledge of LLM on SPICE netlist language, we input the technology independent device (i.e., MOSFET) description in SPICE language format and let LLM explain the MOSFET information. The technology independent description of a MOSFET includes name, terminal connections, and the type of MOSFET. Figure 2(a) shows that LLM can understand technology independent MOSFET description in SPICE language format and identify the net connection information at the drain, gate, and source terminals[1].

B. Cluster Constraint

We study the ability of existing LLM on understanding the cluster design constraint format including multiple devices and clusters. We show one of the example studies in Figure 2(b). We task the LLM with summarizing the provided clusters and the devices associated with them for subsequent reasoning tasks. The study's findings indicate that the existing LLM can accurately identify the total number of clusters and the information regarding devices within these clusters.

[1]The [MOSFET format] is "A MOSFET can be described in SPICE format as MOSFET_NAME d:DRAIN g:GATE s:SOURCE MOSFET_TYPE".

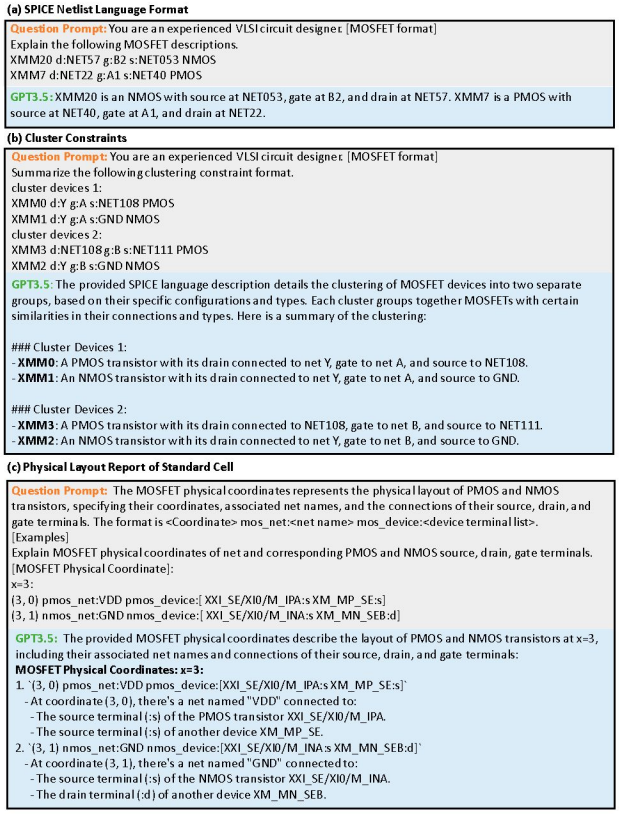


Fig. 2: Standard cell layout design domain knowledge assessment of existing LLM on: (a) SPICE language, (b) cluster constraint, and (c) physical layout.

C. Standard Cell Layout

The standard cell layout includes the placed device locations and the net connection of device terminals. We use coordinates and the corresponding device and its terminals to represent the standard cell layout, as shown in Figure 2(c). We study the capability of LLM in understanding the location of placed devices and transistor terminal connections for the standard cell layout design optimization task. In one of the study examples in Figure 2(c), the LLM successfully explains the placed MOSFETs and their terminals at each coordinate.

In summary, the existing LLM (i.e., GPT3.5) has the capability to understand the netlist topology through SPICE language, device cluster constraint, and device placement, and the terminal connections in standard cell layout description for generating better device cluster constraints for PPA and routability debugging.

III. LLM for Standard Cell Layout Design Optimization

We introduce the details of LLM for standard cell layout design optimization, which leverages the natural language and reasoning ability of LLM to generate high-quality cluster constraints to optimize the cell-level PPA and debug routability issues. The overall flow is outlined in Section III-A, and Figure 3. Section III-B introduces the developed netlist tools, which provide accurate sub-circuit retrieval from the netlist and cluster evaluation, for assisting LLM on layout design optimization task. Finally, we discuss the application of ReAct [5] in Section III-C.

A. Flow Overview

The proposed LLM for standard cell layout design optimization comprises the following components: knowledge extraction

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:28:44 UTC from IEEE Xplore. Restrictions apply.

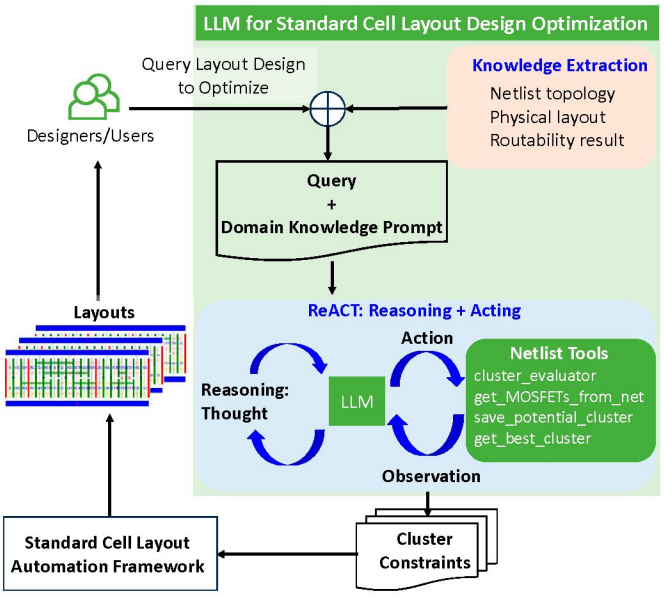


Fig. 3: Overview of LLM for Standard cell layout design optimization flow

to initiate queries and provide domain knowledge prompts, netlist tools to assist the LLM in generating valid cluster constraints, and reasoning and action in ReAct [5] for exploring high-quality cluster candidates for the PPA/routability of layout design.

Firstly, designers input the initial layout and its corresponding cluster constraints. The knowledge extraction is used to create the domain knowledge prompt with netlist topology in technology independent descriptions of MOSFETs, initial cluster constraints, standard cell layout, and guidance from designers' expertise as described in Section II. Then, the ReAct prompting method allows LLM to perform dynamic reasoning to create and adjust plans for acting (reason to act), while also interacting with the netlist tools (i.e., grouping MOSFET, evaluating clusters, etc.) to incorporate additional information into reasoning (act to reason). These cluster constraints can be fed into NVCell [3] to generate layouts. Designers can repeat this process until the PPA meets the requirements without rouability issues.

B. Netlist Tools

We have developed a set of netlist tools to assist LLM in generating valid cluster constraints and correctly retrieving sub-circuits in the ReAct reasoning and action loop. The netlist tools include a cluster evaluator, a function to retrieve group devices from nets, a mechanism to save potential clusters, and a function to obtain the best cluster result.

Cluster evaluator: This tool is used to evaluate the quality of the generated cluster result using simple cluster score to capture the potential diffusion sharing and common gate in the layout. Here, we use the simple cluster score for evaluation in ReAct because the turnaround time is too long to launch layout generation to collect accurate cell layout metrics (i.e., CW, TWL, etc.). The simple cluster score is calculated in Equation (1). A larger score means the devices can potentially be placed with more common diffusion sharing and common gate inside each cluster.

$$cluster_score = \sum_{c \in \mathbf{C}} \left(\frac{\sum_{n \in \mathbf{N_c^d}} \lfloor \frac{P_n}{2} \rfloor + \lfloor \frac{N_n}{2} \rfloor}{T_c} + \frac{\sum_{n \in \mathbf{N_c^g}} \min(P_n, N_n)}{T_c} \right) \quad (1)$$

Where c is a cluster in a set of clusters, $\mathbf{C}$. $\mathbf{N_c^d}$ and $\mathbf{N_c^g}$ represent the set of nets at diffusion (i.e., source or drain terminals) and gate in cluster c. P_n and N_n represent the number of net n at PMOS and NMOS terminals, respectively. T_c is the number of transistors in cluster c. For potential diffusion sharing in a cluster, we calculate the number of same net pairs at diffusion terminals for PMOS and NMOS, respectively. The minimum number of the same gate net of PMOS and NMOS is the potential number of common gates in a cluster.



Fig. 4: An example of ReAct steps of Thought-Action-Observation for standard cell layout design optimization.

Get group devices from nets: This netlist tool returns the group of transistors from an arbitrary number of nets in the netlist. LLM can use this tool to search and explore potential device clusters.

Save potential cluster: This tool returns the current clusters and cluster score after LLM inputs a new potential cluster. The duplicated devices in different clusters are fixed based on the number of shared nets of these duplicated devices in each cluster.

Get best cluster result: This tool returns the cluster result with the best simple cluster score (i.e., Equation (1)). It helps LLM revert back or restart the search from the previous best cluster result when it is stuck in the searching potential cluster phase.

C. ReAct: Reason + Act Loop

We enable LLMs to function as autonomous circuit design agent for reasoning and acting with the netlist tools through **ReAct** prompting mechanism [5]. In ReAct, the LLM initiates the generation of subsequent steps with Thought, Action, and Observation components. The action is querying one of the netlist tools described in Section III-B. The output of the queried netlist tool from action becomes the observation in the prompt. The agent continues the reasoning and acting traces until selecting the "Final Answer" action. Figure 4 shows an example of optimizing the cell area of a standard cell. Here, the agent starts with querying the group of devices connected to NET027 to explore

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:28:44 UTC from IEEE Xplore. Restrictions apply.

good clusters incrementally to reduce the diffusion break for area reduction since NET027 is one of the high connection nets in the netlist topology and abutted to the diffusion break dummy device in the physical layout. Finally, the agent successfully generates high-quality cluster result through reasoning and leveraging the netlist tools traces in ReAct.

IV. Experimental Results

Our work is implemented with Python and LangChain [7]. We conduct all experiments with gpt-3.5-turbo-16k-0613 as the LLM through OpenAI APIs [8]. We set the sampling temperature of LLM to 0.1. For ReAct prompting, we restrict the LLM to a maximum of 15 iterations of Thought-Action-Observation.

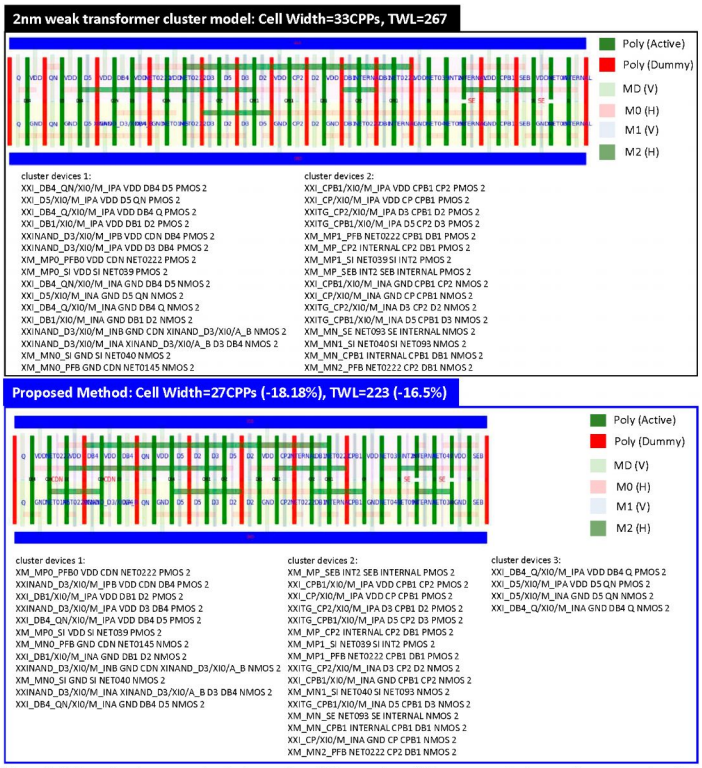


Fig. 5: The cluster constraints and cell layouts of ***2nm* weak transformer cluster model** [4], and proposed method of Seq3 cell. The proposed method successfully reduce the CW, and TWL by 18.18%, and 16.48%, respectively.

TABLE I: CW (CPPs) table of the 17 complex sequential cells in 2*nm* of transformer cluster model [4], simulated annealing (SA), and the proposed method. The SA algorithm is implemented base on [6]. #Devs=number of transistors. Opt.(a)/(b)=Optimize layouts from (a)/(b). X=unroutable. V=Fix routability. Success(%)=the percentage of LVS/DRC clean cell layouts.

Cell	#Devs	Transformer Cluster [4]		SA		Proposed Method			
								Impr. (%)	
		(a) 2nm weak	(b) 5nm	Opt. (a)	Opt. (b)	Opt. (a)	Opt. (b)	Over (a)	Over (b)
Seq1	40	26	31	X	31	25	25	3.85	19.35
Seq2	60	X	41	X	X	39	39	V	4.88
Seq3	40	33	26	27	26	27	26	18.18	0.00
Seq4	38	27	25	X	25	25	25	7.41	0.00
Seq5	36	23	22	26	22	22	22	4.35	0.00
Seq6	36	22	25	22	22	22	22	0.00	12.00
Seq7	34	20	20	20	20	20	20	0.00	0.00
Seq8	38	25	25	25	25	24	25	4.00	0.00
Seq9	32	19	19	19	19	19	19	0.00	0.00
Seq10	34	20	21	20	20	20	20	0.00	4.76
Seq11	40	26	26	25	28	25	25	3.85	3.85
Seq12	38	24	27	26	28	24	24	0.00	11.11
Seq13	56	X	X	X	X	41	40	V	V
Seq14	44	X	36	X	X	34	34	V	5.56
Seq15	42	X	32	X	X	31	31	V	3.13
Seq16	56	42	40	X	40	35	35	16.67	12.50
Seq17	42	25	25	25	25	25	25	0.00	0.00
Success (%)		76.50	94.10	58.80	76.50	100	100	-	-

We conduct extensive studies on the cell area and routability using 17 complex sequential standard cells in industrial 2*nm* technology node. Due to a lack of cell layout data in the early stage development of cell library in 2*nm*, we trained (a) ***2nm* weak transformer cluster model** [4] using 124 LVS/DRC clean cell layouts, and (b) ***5nm* transformer cluster model** [4] using 512 LVS/DRC clean cell layouts as the baselines. We implemented a simulated annealing algorithm (SA), which is based on the modified Lam annealing scheduler [6] that required no hyper parameter tuning, for comparing the efficiency of the proposed method. In SA, we sample arbitrary 1 to k nets considering the weights of nets and querying a group of devices from **Get group devices from nets** tool for each *action*. The unrouted nets, and nets abutted to diffusion break dummy device have higher weights. Then, this queried group of devices will be accepted or rejected to be saved using **Save potential cluster** tool based on the temperature and the delta simple cluster score. Finally, we apply the proposed method, and SA to optimize the cell area and routability from the initial cluster constraints and cell layouts of (a), and (b) on the selected unseen 17 complex sequential cells. For the proposed method, we launch 10 optimization runs for each cell and select valid cluster results to generate cell layout using NVCell [3].

Table I shows the CW (CPPs) of the selected 17 sequential cells of transformer cluster method [4], SA, and the proposed method. SA fails to optimize the cell area and fix the routability because the simple cluster score can not fully capture diffusion sharing, and routability in the layout. It requires more holistic and efficient methodology for cluster exploration than a naive net sampling-based approach. Compared to (a), the proposed method can not only reduce up to 18.18% of CW, but also improve the success rate from 76.5% to 100%. In addition, the proposed method reduces up to 19.35% of CW, and improves the success rate from 94.10% to 100% when optimizing the initial cell layout from (b). Figure 5 shows the layouts and generated cluster constraints of (a), and using proposed method to optimize from (a) of Seq3 cell. The proposed method achieves 18.18% reduction in CW, and 16.48% reduction in TWL.

In summary, excluding the fix routability cells, the proposed method achieves 4.48%, and 4.82% reduction in cell area on average over (a), and (b). We successfully demonstrate that the proposed method incorporates holistic information from the netlist and layout of complex sequential cells to conduct efficient cluster exploration with a simple cluster score for the standard cell layout design optimization task.

V. Conclusion

We propose a novel, efficient, and the first LLM for standard cell layout design optimization methodology to generate high-quality cluster constraints to optimize the cell layout PPA and debug the routability with the guidance of designers' expertise and ReAct [5] prompting techniques. We have demonstrated that the proposed method achieves up to 19.4% smaller cell area, and generates 100% LVS/DRC clean cell layouts on the selected complex sequential cell benchmark in 2*nm*. This research not only provides a novel autonomous LLM agent for standard cell layout design optimization and debugging but also introduces the new application of using LLM assistance for optimization in the EDA field for further exploration.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:28:44 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Haoxing Ren and Matthew Fojtik. Nvcell: Standard cell layout in advanced technology nodes with reinforcement learning. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pages 1291–1294. IEEE, 2021.

[2] Pascal Van Cleeff, Stefan Hougardy, Jannik Silvanus, and Tobias Werner. Bonncell: Automatic cell layout in the 7-nm era. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 39(10):2872–2885, 2019.

[3] Chia-Tung Ho, Alvin Ho, Matthew Fojtik, Minsoo Kim, Shang Wei, Yaguang Li, Brucek Khailany, and Haoxing Ren. Nvcell 2: Routability-driven standard cell layout in advanced nodes with lattice graph routability model. In *Proceedings of the 2023 International Symposium on Physical Design*, pages 44–52, 2023.

[4] Chia-Tung Ho, Ajay Chandna, David Guan, Alvin Ho, Minsoo Kim, Yaguang Li, and Haoxing Ren. Novel transformer model based clustering method for standard cell design automation. In *Proceedings of the 2024 International Symposium on Physical Design*, pages 195–203, 2024.

[5] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

[6] Vincent A Cicirello. On the design of an adaptive simulated annealing algorithm. In *Proceedings of the international conference on principles and practice of constraint programming first workshop on autonomous search*, 2007.

[7] Harrison Chase. LangChain, October 2022.

[8] OpenAI. Openai models api. 2023.

VI. APPENDIX

We present examples of netlist topology, physical layout, and routability report prompts from proposed knowledge extraction component as mentioned in Fig. 4. We describe the netlist topology and physical layout prompts of the simple cell example (i.e., OA333X1) in Fig. 4. For the routability report prompt, we select the Seq13 in Table I for demonstration, as there are no unrouted nets in the OA333X1 layout depicted in Fig. 4.

A. Netlist topology prompt

The netlist topology prompt consists of MOSFET connection and description, as well as previous cluster constraints, as shown in Fig. 6. In the MOSFET connection and description, each MOSFET is described using the technology-independent device description format introduced in Section II. For the previous cluster constraints, we present them in JSON BLOB format with the action labeled as "Final Answer". At the end, we provide the simple cluster score and the number of clusters resulting from the previous cluster constraints.

With the netlist topology prompt information, the LLM agent can understand the netlist connections of each device, previous cluster constraints, and the simple cluster score of previous cluster constraints for the following reasoning and action for cell layout design optimization.

MOSFET Connection and Description

```
[Circuit Information begin]:
CELLNAME OA333X1
== MOSFET_NAME SOURCE GATE DRAIN MOSFET_TYPE ==
XMM20 s:Y g:NET027 d:VDD PMOS
XMM19 s:NET087 g:B2 d:NET058 PMOS
XMM23 s:VDD g:A3 d:VDD PMOS
XMM13 s:NET040 g:C3 d:VDD PMOS
XMM14 s:NET027 g:A1 d:NET084 PMOS
XMM16 s:NET086 g:C2 d:NET040 PMOS
XMM12 s:NET027 g:B1 d:NET087 PMOS
XMM11 s:NET058 g:B2 d:NET041 PMOS
XMM17 s:NET027 g:C1 d:NET086 PMOS
XMM3 s:NET038 g:A3 d:VDD PMOS
XMM10 s:NET041 g:B3 d:VDD PMOS
XMM2 s:NET084 g:A2 d:NET038 PMOS
XMM22 s:GND g:NET027 d:Y NMOS
XMM18 s:NET068 g:A3 d:NET068 NMOS
XMM21 s:NET068 g:B2 d:NET068 NMOS
XMM4 s:NET015 g:B1 d:NET068 NMOS
XMM1 s:GND g:C3 d:NET015 NMOS
XMM8 s:NET068 g:A2 d:NET027 NMOS
XMM9 s:NET068 g:A3 d:NET027 NMOS
XMM6 s:NET015 g:B3 d:NET068 NMOS
XMM7 s:NET068 g:A1 d:NET027 NMOS
XMM15 s:GND g:C1 d:NET015 NMOS
XMM5 s:NET015 g:B2 d:NET068 NMOS
XMM0 s:GND g:C2 d:NET015 NMOS
```

Previous Cluster Constraints

```
[Previous Cluster Result]:
AI: Action:
```
{"action": "Final Answer",
"action_input":"
cluster devices 1:
XMM20 s:Y g:NET027 d:VDD PMOS
XMM19 s:NET087 g:B2 d:NET058 PMOS
XMM14 s:NET027 g:A1 d:NET084 PMOS
XMM12 s:NET027 g:B1 d:NET087 PMOS
XMM11 s:NET058 g:B2 d:NET041 PMOS
XMM17 s:NET027 g:C1 d:NET086 PMOS
XMM22 s:GND g:NET027 d:Y NMOS
XMM21 s:NET068 g:B2 d:NET068 NMOS
XMM8 s:NET068 g:A2 d:NET027 NMOS
XMM9 s:NET068 g:A3 d:NET027 NMOS
XMM7 s:NET068 g:A1 d:NET027 NMOS
XMM5 s:NET015 g:B2 d:NET068 NMOS
"
}
```
Cluster score of shared source, drain, and gate terminals: 0.004166666666666721
Number of clusters: 1
```

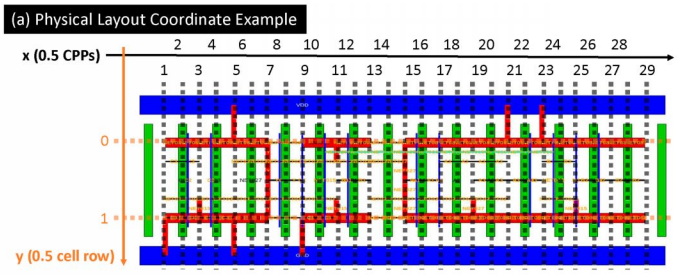
Fig. 6: An example of extracted netlist topology prompt of the simple cell example in Fig. 4.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:28:44 UTC from IEEE Xplore. Restrictions apply.

B. Physical layout prompt

We design the physical layout prompt with the placed device locations and net connection of device terminals for LLM to compile the netlist topology and layout together for ReAct steps of Thought-Action-Observation.

Fig. 7 shows the example of extracted physical layout prompt of OA333X1 layout. From the Fig. 7(a), the unit of x coordinate is the half contacted-poly-pitch (CPP), and the unit of y coordinate is half cell row. As a result, there are 29 columns and 2 rows in OA333x1 layout in the extracted coordinate based physical layout prompt in Fig. 7(b). For each coordinate, we show the net name, placed device, and the terminals (i.e., source, drain, gate) of the placed device. The net name, and device are dummy when there are no devices in the netlist being placed at the coordinate. Here, the column-based physical layout report format helps LLM identify the common gate and diffusion connections of PMOS and NMOS.



(b) Physical Layout Prompt

```
[OA333X1 MOSFET Physical Coordinate Report of Previous Cluster Result]:
[MOSFET Physical Coordinate]:
x=1:
(1, 0) pmos_net:NET086 pmos_device:[ XMM16:s]
(1, 1) nmos_net:GND nmos_device:[ XMM0:s]
x=2:
(2, 0) pmos_net:C2 pmos_device:[XMM16:g]
(2, 1) nmos_net:C2 nmos_device:[XMM0:g]
x=3:
(3, 0) pmos_net:NET040 pmos_device:[ XMM13:s XMM16:d]
(3, 1) nmos_net:NET015 nmos_device:[ XMM1:d XMM0:d]
x=4:
(4, 0) pmos_net:C3 pmos_device:[XMM13:g]
(4, 1) nmos_net:C3 nmos_device:[XMM1:g]
x=5:
(5, 0) pmos_net:VDD pmos_device:[ XMM20:d XMM13:d]
(5, 1) nmos_net:GND nmos_device:[ XMM22:s XMM1:s]
x=6:
(6, 0) pmos_net:NET027 pmos_device:[XMM20:g]
(6, 1) nmos_net:NET027 nmos_device:[XMM22:g]
x=7:
(7, 0) pmos_net:Y pmos_device:[ XMM20:s]
(7, 1) nmos_net:Y nmos_device:[ XMM22:d]
x=8:
(8, 0) pmos_net:dummy pmos_device:[dummy:g]
(8, 1) nmos_net:dummy nmos_device:[dummy:g]
x=9:
(9, 0) pmos_net:NET086 pmos_device:[ XMM17:d]
(9, 1) nmos_net:GND nmos_device:[ XMM15:s]
x=10:
(10, 0) pmos_net:C1 pmos_device:[XMM17:g]
(10, 1) nmos_net:C1 nmos_device:[XMM15:g]
x=11:
(11, 0) pmos_net:NET027 pmos_device:[ XMM12:s XMM17:s]
(11, 1) nmos_net:NET015 nmos_device:[ XMM4:s XMM15:d]
x=12:
(12, 0) pmos_net:B1 pmos_device:[XMM12:g]
(12, 1) nmos_net:B1 nmos_device:[XMM4:g]
x=13:
(13, 0) pmos_net:NET087 pmos_device:[ XMM19:s XMM12:d]
(13, 1) nmos_net:NET068 nmos_device:[ XMM21:d XMM4:d]
x=14:
(14, 0) pmos_net:B2 pmos_device:[XMM19:g]
(14, 1) nmos_net:B2 nmos_device:[XMM21:g]
x=15:
(15, 0) pmos_net:NET058 pmos_device:[ XMM19:d XMM11:s]
(15, 1) nmos_net:NET068 nmos_device:[ XMM21:s XMM5:d]
x=16:
(16, 0) pmos_net:B2 pmos_device:[XMM11:g]
(16, 1) nmos_net:B2 nmos_device:[XMM5:g]
x=17:
(17, 0) pmos_net:NET041 pmos_device:[ XMM11:d]
(17, 1) nmos_net:NET015 nmos_device:[ XMM5:s]
x=18:
(18, 0) pmos_net:dummy pmos_device:[dummy:g]
(18, 1) nmos_net:dummy nmos_device:[dummy:g]
x=19:
(19, 0) pmos_net:NET027 pmos_device:[ XMM14:s]
(19, 1) nmos_net:NET027 nmos_device:[ XMM7:d]
x=20:
(20, 0) pmos_net:A1 pmos_device:[XMM14:g]
(20, 1) nmos_net:A1 nmos_device:[XMM7:g]
x=21:
(21, 0) pmos_net:NET084 pmos_device:[ XMM14:d XMM2:s]
(21, 1) nmos_net:NET068 nmos_device:[ XMM8:s XMM7:s]
x=22:
(22, 0) pmos_net:A2 pmos_device:[XMM2:g]
(22, 1) nmos_net:A2 nmos_device:[XMM8:g]
x=23:
(23, 0) pmos_net:NET038 pmos_device:[ XMM3:s XMM2:d]
(23, 1) nmos_net:NET027 nmos_device:[ XMM8:d XMM9:d]
x=24:
(24, 0) pmos_net:A3 pmos_device:[XMM3:g]
(24, 1) nmos_net:A3 nmos_device:[XMM9:g]
x=25:
(25, 0) pmos_net:VDD pmos_device:[ XMM23:d XMM3:d]
(25, 1) nmos_net:NET068 nmos_device:[ XMM18:s XMM9:s]
x=26:
(26, 0) pmos_net:A3 pmos_device:[XMM23:g]
(26, 1) nmos_net:A3 nmos_device:[XMM18:g]
x=27:
(27, 0) pmos_net:VDD pmos_device:[ XMM23:s XMM10:d]
(27, 1) nmos_net:NET068 nmos_device:[ XMM18:d XMM6:d]
x=28:
(28, 0) pmos_net:B3 pmos_device:[XMM10:g]
(28, 1) nmos_net:B3 nmos_device:[XMM6:g]
x=29:
(29, 0) pmos_net:NET041 pmos_device:[ XMM10:s]
(29, 1) nmos_net:NET015 nmos_device:[ XMM6:s]
```

Fig. 7: An example of extracted physical layout prompt of the simple cell example in Fig. 4.

C. Routability report prompt

We select the Seq13 in Table I for describing the routability report prompt format since there are no unrouted nets in OA333X1 layout as shown in Fig. 8. In the routability report, we provide information about the unrouted nets, the corresponding pairs of x-coordinates of net terminals, and the placed devices inside the unrouted region. These placed devices within the unrouted region offer insight into routing congestion and required transistor pin access, assisting LLM in identifying potential good cluster constraints to improve routability. For example, if routing congestion or pin density is too high in an unrouted net region, leveraging common transistor terminal sharing across PMOS and NMOS, as well as diffusion sharing, can reduce pin density and routing resource usage by creating cluster constraints that consider the high connection nets or problematic nets of transistor pins in an unrouted net region.

```
[Route Evaluation]: Failed to route using Previous Cluster Result
[Failed to route net NET206]:
Failed to route net region: from x=6 to x=14
devices in the unrouted region:
XXI_SE/XI0/M_IPA VDD SE SEB PMOS
XM_MP_EB NET247 EB NET234 PMOS
XM_MP_E NET235 E NET234 PMOS
XM_MP_D5 VDD D5 NET235 PMOS
XXI_SE/XI0/M_INA GND SE SEB NMOS
XM_MN_EB NET202 EB NET206 NMOS
XM_MN_E NET190 E NET206 NMOS
XM_MN_D GND D NET190 NMOS

[Failed to route net CPB1]:
Failed to route net region: from x=42 to x=44
devices in the unrouted region:
XXITG_CP2/XI0/M_IPA D3 CPB1 D2 PMOS
XXITG_CPB1/XI0/M_IPA D5 CP2 D3 PMOS
XXITG_CP2/XI0/M_INA D3 CP2 D2 NMOS
XXITG_CPB1/XI0/M_INA D5 CPB1 D3 NMOS
Failed to route net region: from x=44 to x=56
devices in the unrouted region:
XXI_DB1/XI0/M_IPA VDD DB1 D2 PMOS
XXITG_CP2/XI0/M_IPA D3 CPB1 D2 PMOS
XM_MP1_PFB NET029 CPB1 DB1 PMOS
XM_MP0_PFB1 VDD D2 NET029 PMOS
XM_MP0_PFB0 VDD CDN NET029 PMOS
XM_MP_CP2 NET182 CP2 DB1 PMOS
XXI_DB1/XI0/M_INA GND DB1 D2 NMOS
XXITG_CP2/XI0/M_INA D3 CP2 D2 NMOS
XM_MN2_PFB NET0222 CP2 DB1 NMOS
XM_MN0_PFB GND CDN NET0181 NMOS
XM_MN1_PFB NET0181 D2 NET0222 NMOS
XM_MN_CPB1 NET182 CPB1 DB1 NMOS
```

Fig. 8: An example of extracted routability report prompt of Seq13 in Table I.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:28:44 UTC from IEEE Xplore. Restrictions apply.